Exercise 1: Control Structures

```
CREATE TABLE Customers (
    CustomerID NUMBER PRIMARY KEY,
    Name VARCHAR2(50),
    Age NUMBER,
    Balance NUMBER,
    IsVIP VARCHAR2(5)
);

CREATE TABLE Loans (
    LoanID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    InterestRate NUMBER,
    DueDate DATE
);


INSERT INTO Customers VALUES (1,'John',65,15000,'FALSE');
INSERT INTO Customers VALUES (2,'Smith',45,8000,'FALSE');

INSERT INTO Loans VALUES (101,1,10,TO_DATE('15-JUL-2026','DD-MON-YYYY'));
INSERT INTO Loans VALUES (102,2,12,TO_DATE('20-JUL-2026','DD-MON-YYYY'));

COMMIT;

SELECT * FROM Loans;
```

```
SELECT * FROM Loans;

BEGIN
    FOR cust IN (
        SELECT c.CustomerID, l.LoanID
        FROM Customers c
        JOIN Loans l
        ON c.CustomerID = l.CustomerID
        WHERE c.Age > 60
    )
    LOOP
        UPDATE Loans
        SET InterestRate = InterestRate - 1
        WHERE LoanID = cust.LoanID;
    END LOOP;

    COMMIT;
END;
/

SELECT * FROM Loans;
```

Output:

```
    LOANID CUSTOMERID INTERESTRATE DUEDATE
---------- ---------- ------------ ------------------
       101          1           10 15-JUL-26
       102          2           12 20-JUL-26

    LOANID CUSTOMERID INTERESTRATE DUEDATE
---------- ---------- ------------ ------------------
       101          1            9 15-JUL-26
       102          2           12 20-JUL-26
```

```
SELECT * FROM Loans;

BEGIN
    FOR cust IN (
        SELECT CustomerID
        FROM Customers
        WHERE Balance > 10000
    )
    LOOP
        UPDATE Customers
        SET IsVIP = 'TRUE'
        WHERE CustomerID = cust.CustomerID;
    END LOOP;

    COMMIT;
END;
/

SELECT * FROM Loans;
```

Output:

```
    LOANID CUSTOMERID INTERESTRATE DUEDATE
---------- ---------- ------------ ------------------
       101          1           10 15-JUL-26
       102          2           12 20-JUL-26

    LOANID CUSTOMERID INTERESTRATE DUEDATE
---------- ---------- ------------ ------------------
       101          1           10 15-JUL-26
       102          2           12 20-JUL-26
```

```sql
SELECT * FROM Loans;

SET SERVEROUTPUT ON;

BEGIN
    FOR loan_rec IN (
        SELECT c.Name, l.LoanID, l.DueDate
        FROM Customers c
        JOIN Loans l
        ON c.CustomerID = l.CustomerID
        WHERE l.DueDate BETWEEN SYSDATE AND SYSDATE + 30
    )
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            'Reminder: Dear ' || loan_rec.Name ||
            ', your loan is due on ' ||
            TO_CHAR(loan_rec.DueDate,'DD-MON-YYYY')
        );
    END LOOP;
END;
/

SELECT * FROM Loans;
```

Output:

```
    LOANID CUSTOMERID INTERESTRATE DUEDATE
---------- ---------- ------------ ------------------
       101          1           10 15-JUL-26
       102          2           12 20-JUL-26
Reminder: Dear John, your loan is due on 15-JUL-2026
Reminder: Dear Smith, your loan is due on 20-JUL-2026

    LOANID CUSTOMERID INTERESTRATE DUEDATE
---------- ---------- ------------ ------------------
       101          1           10 15-JUL-26
       102          2           12 20-JUL-26
```

Exercise 3: Stored Procedures

```
CREATE TABLE Accounts (
    AccountID NUMBER PRIMARY KEY,
    AccountType VARCHAR2(20),
    Balance NUMBER
);

CREATE TABLE Employees (
    EmployeeID NUMBER PRIMARY KEY,
    EmployeeName VARCHAR2(50),
    Department VARCHAR2(30),
    Salary NUMBER
);

INSERT INTO Accounts VALUES (101, 'Savings', 10000);
INSERT INTO Accounts VALUES (102, 'Savings', 20000);
INSERT INTO Accounts VALUES (103, 'Current', 15000);

INSERT INTO Employees VALUES (1, 'John', 'IT', 50000);
INSERT INTO Employees VALUES (2, 'Smith', 'IT', 60000);
INSERT INTO Employees VALUES (3, 'David', 'HR', 45000);

COMMIT;
```

```
SELECT * FROM Accounts;

CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'Savings';

    COMMIT;
END;
/

BEGIN
    ProcessMonthlyInterest;
END;
/

SELECT * FROM Accounts;
```

Output:

```
 ACCOUNTID ACCOUNTTYPE          BALANCE
---------- -------------------- ----------
       101 Savings              10000
       102 Savings              20000
       103 Current              15000

 ACCOUNTID ACCOUNTTYPE          BALANCE
---------- -------------------- ----------
       101 Savings              10100
       102 Savings              20200
       103 Current              15000
```

```
SELECT * FROM Employees;

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
    p_department IN VARCHAR2,
    p_bonus_percent IN NUMBER
)
IS
BEGIN
    UPDATE Employees
    SET Salary = Salary + (Salary * p_bonus_percent / 100)
    WHERE Department = p_department;

    COMMIT;
END;
/

BEGIN
    UpdateEmployeeBonus('IT', 10);
END;
/

SELECT * FROM Employees;
```

Output:

```
EMPLOYEEID EMPLOYEENAME
---------- --------------------------------------------------
DEPARTMENT                     SALARY
------------------------------ ----------
     1 John
IT                             50000

     2 Smith
IT                             60000

     3 David
HR                             45000


EMPLOYEEID EMPLOYEENAME
---------- --------------------------------------------------
DEPARTMENT                     SALARY
------------------------------ ----------
     1 John
IT                             55000

     2 Smith
IT                             66000

     3 David
HR                             45000
```

```sql
SELECT * FROM Accounts;

CREATE OR REPLACE PROCEDURE TransferFunds(
    p_from_account IN NUMBER,
    p_to_account IN NUMBER,
    p_amount IN NUMBER
)
IS
    v_balance NUMBER;
BEGIN

    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from_account;

    IF v_balance >= p_amount THEN

        UPDATE Accounts
        SET Balance = Balance - p_amount
        WHERE AccountID = p_from_account;

        UPDATE Accounts
        SET Balance = Balance + p_amount
        WHERE AccountID = p_to_account;

        COMMIT;

        DBMS_OUTPUT.PUT_LINE('Transfer Successful');

    ELSE

        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');

    END IF;

END;
/

SET SERVEROUTPUT ON;

BEGIN
    TransferFunds(101, 102, 5000);
END;
/

SELECT * FROM Accounts;
```

Output:

```
 ACCOUNTID ACCOUNTTYPE         BALANCE
---------- -------------------- ----------
       101 Savings           10000
       102 Savings           20000
       103 Current           15000
Transfer Successful

 ACCOUNTID ACCOUNTTYPE         BALANCE
---------- -------------------- ----------
       101 Savings            5000
       102 Savings           25000
       103 Current           15000
```